Personalized E-Commerce Search

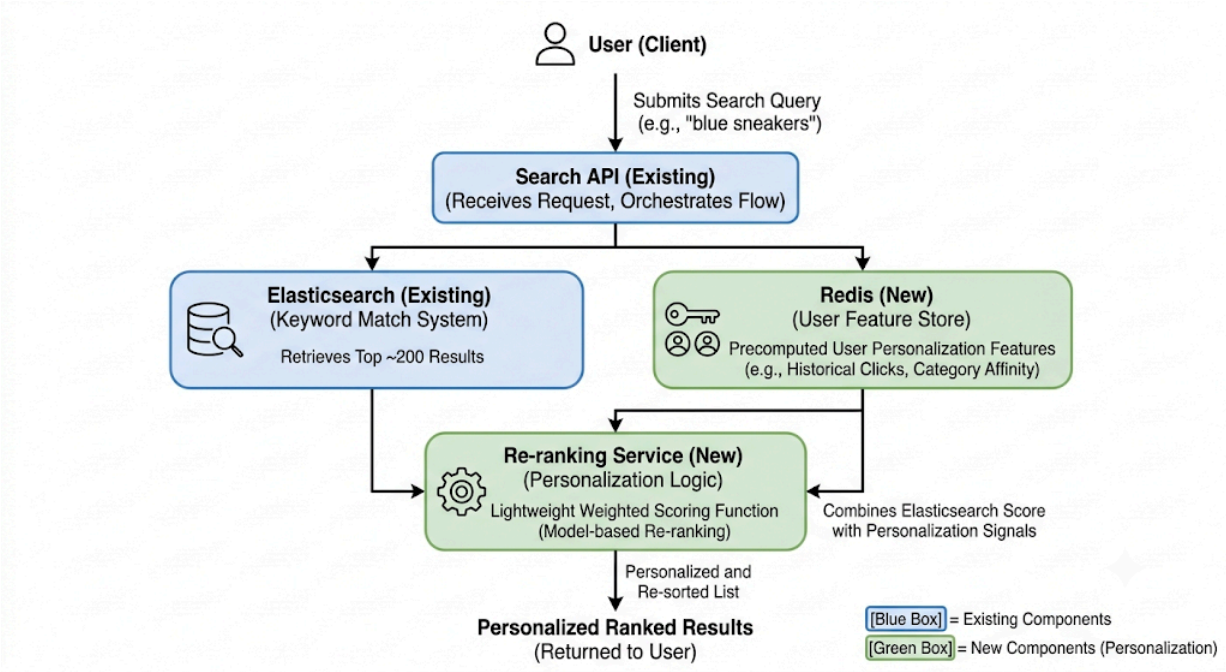
ML System Design Assignment

Saksham Singhal

1 Executive Summary

Goal	Personalize product search using customer purchase and search history without modifying the existing keyword search engine.
Approach	Build simple aggregation tables offline from historical user behavior. Re-rank the top keyword search results using a lightweight scoring formula. Fall back to the original keyword ranking whenever personalization data is unavailable or stale.
Components	Nightly batch pipeline to generate aggregation tables. Redis cache for low-latency lookups. Lightweight re-ranking service. Existing keyword search engine (unchanged).
Metrics	**Offline (before launch)** Evaluate ranking quality on historical purchase data using NDCG@10, which measures whether relevant products appear near the top of the results, and MRR, which measures how quickly the first relevant product is shown. **Online (after launch)** Measure Click-Through Rate (CTR), Conversion Rate, and Revenue per Search to evaluate user engagement and the overall business impact of personalization. **Guardrails** Monitor p95/p99 latency to ensure search remains fast, zero-result rate to avoid empty search responses, and result diversity to provide users with a balanced and varied set of products.

Diagram 1 : Quick Overview: High-Level Online Search Flow



Source: *Diagram created by the author with assistance from Google Gemini.*

2 Design Philosophy

The goal of this design is to keep the solution simple, practical, and easy to maintain. Instead of introducing complex models or real-time infrastructure, I have focused on using lightweight personalization based on historical user behavior. The objective is to improve search quality while keeping the existing keyword search system unchanged.

This report is divided into two parts. The first explains the personalization logic and the signals used for re-ranking. The second describes how this logic integrates with the current search system without affecting its existing functionality.

3 Assumptions

The following assumptions are made to keep the scope of the solution clear and aligned with the assignment requirements.

- **Personalization is applied only for re-ranking.** The existing keyword search engine is assumed to retrieve relevant candidate products, while the personalization layer only changes their ranking based on historical behavior.
- **Historical user behavior is predictive of future preferences.** Previous searches and purchases, especially repeat purchases, are treated as strong indicators of future intent.

- **Guest users receive session-level personalization only.** Since long-term history is unavailable, personalization is limited to interactions within the current browsing session.
- **Search queries are normalized before processing.** Basic preprocessing such as lowercasing, trimming, and typo/stemming normalization is applied to ensure consistent aggregation and feature lookup.
- **The personalization layer must preserve search latency.** The existing search API is assumed to operate within a p95 latency of approximately 100–150 ms. Personalization should introduce only minimal additional latency.
- **Behavioral features are generated offline.** Search logs and order history are processed through nightly batch jobs, while request-time personalization relies only on lightweight lookups and score computation.
- **Sufficient historical search and order data is available.** The system assumes that historical logs contain enough user interactions to generate reliable personalization signals through offline aggregation.

4 Part 1 : The Personalisation / Recommendation Engine

4.1 Personalization Signals

The personalization layer uses historical search and purchase behavior to improve the ranking of products retrieved by the existing keyword search engine. The following signals are used:

- **Query >> Product:** For each normalized search query, aggregate user clicks and purchases. Purchases receive higher weight than clicks because they indicate stronger user intent.
- **Product >> Product:** Track products that are frequently purchased or clicked together. This helps recommend related products and provides useful signals for newly introduced products.
- **User >> Affinity:** Build a user profile based on preferred categories and brands using recent purchase and search history, with higher weight given to recent interactions.
- **Repeat Purchase History:** If a user has previously purchased the same product for a similar query, it is treated as the strongest personalization signal.

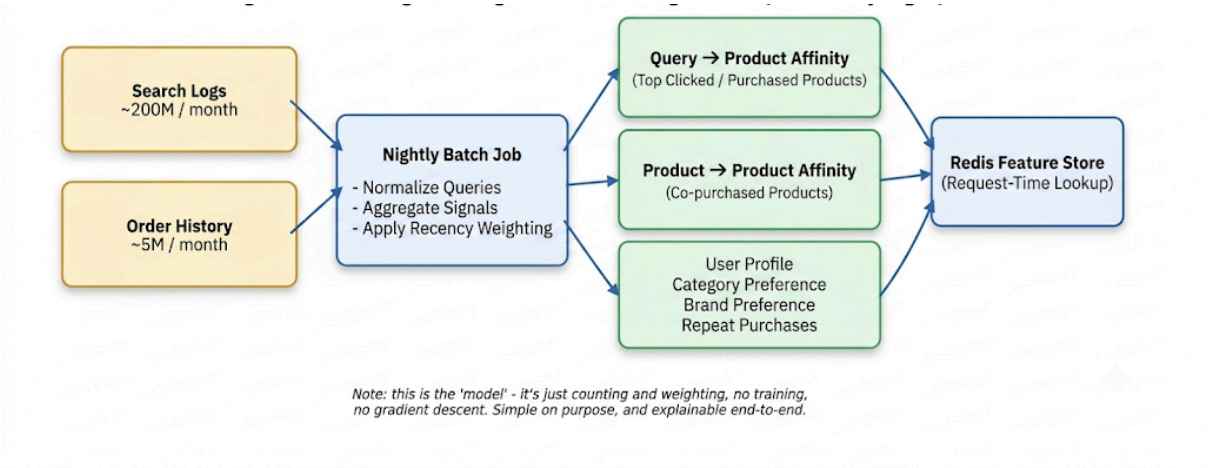
These behavioral signals are generated offline and stored for fast retrieval during search requests.

4.2 Connecting the Current Query to Historical Behaviour

When a user submits a search query, Elasticsearch first retrieves the top keyword-matched candidate products. The personalization layer then combines population-level behavior with user-specific history to re-rank these candidates.

For logged-in users, query popularity, category affinity, and repeat-purchase history are used. For guest users, personalization relies only on query-level popularity and interactions within the current browsing session.\

Diagram 2 : Nightly Offline Feature Generation Pipeline



Source: *Diagram created by the author with assistance from Google Gemini.*

4.3 Ranking Formula

Each candidate product receives a final ranking score by combining keyword relevance with personalization signals.

Final Score =
0.70 × BM25 + 0.15 × Query Popularity + 0.10 × User Affinity + 0.05 × Repeat Purchase

All features are normalized to a common scale before scoring. Keyword relevance receives the highest weight because it remains the primary retrieval signal, while personalization signals are used to improve the ranking of already relevant products.

The weights shown above are **initial heuristic values** and can be tuned using offline evaluation (e.g., NDCG@10 and MRR) before deployment and further refined through online A/B testing.

4.4 Offline vs Online Processing

Feature	Source	Computed	Refresh
BM25	Elasticsearch	Online	Real-time
Query Popularity	Search Logs	Offline	Nightly
User Affinity	Order History	Offline	Nightly
Repeat Purchase	Order History	Offline	Nightly
Session Clicks	Current Session	Online	Real-time

The offline pipeline generates personalization features using historical logs, while request-time processing only performs lightweight lookups and score computation.

4.5 Example

Suppose a user searches for **"running shoes."**

Although the generic product has a slightly higher keyword relevance score, the Nike product achieves a higher final ranking because it aligns better with the user's historical purchases and category preferences. This example illustrates how personalization improves ranking while preserving keyword relevance as the primary retrieval signal.

Product	BM25 (× 0.70)	Query Pop. (× 0.15)	User Affin. (× 0.10)	Repeat (× 0.05)	Final Score
Nike Air Zoom Pegasus	0.85	0.70	0.80	1.00	0.83
Generic Store-Brand Shoe	0.90	0.20	0.10	0.00	0.67

```
Where Nike's 0.83 comes from:
   0.70 × 0.85 = 0.595
   0.15 × 0.70 = 0.105
   0.10 × 0.80 = 0.080
   0.05 × 1.00 = 0.050
   ──────────────────
   Total       = 0.830
```

Source: *Illustrative example created by the author with assistance from Claude.*

4.6 Cold Start

New Users: If no historical user data is available, the system falls back to query-level popularity and the default keyword ranking.

New Products: Products without interaction history are ranked using keyword relevance and may optionally receive a small boost from category-level popularity until sufficient behavioral data is collected.

4.7 Evaluation

Offline Evaluation
Historical search and purchase data can be replayed to compare the proposed ranking against the existing keyword ranking. Metrics such as **NDCG@10** and **MRR** are used to measure ranking quality before deployment.

Online Evaluation
After deployment, an A/B test should compare the personalized search experience with the existing system. Primary success metrics include **Click-Through Rate (CTR)**, **Conversion Rate**, and **Revenue per Search**. Guardrail metrics such as **p95/p99 latency**, **zero-result rate**, and **result diversity** ensure that personalization does not negatively impact user experience.

4.8. Alternative Considered

Several alternative approaches were considered but were not selected for the initial implementation.

- **Learning-to-Rank Models:** Models such as Gradient Boosted Trees can improve ranking accuracy but require labeled training data, feature engineering, and additional serving infrastructure. A lightweight rule-based approach is more suitable for an initial deployment.
- **Embedding-based Semantic Search:** Semantic retrieval helps address vocabulary mismatch and semantic similarity. However, the existing keyword search engine already retrieves relevant candidates, making semantic retrieval unnecessary for the current objective of personalization.
- **Real-time Streaming Pipelines:** Streaming systems can continuously update personalization features but introduce additional operational complexity. Nightly batch processing provides sufficient freshness while keeping the architecture simple and maintainable.

5 Part 2 : Integration into the Existing Engineering System

5.1 System Integration

The proposed personalization layer is designed to work on top of the existing keyword search engine without modifying its retrieval logic. Elasticsearch continues to retrieve the top keyword-matched candidate products, while personalization is applied only during the ranking stage.

This approach minimizes implementation effort, preserves the existing search quality, and allows personalization to be enabled or disabled independently of the search engine.

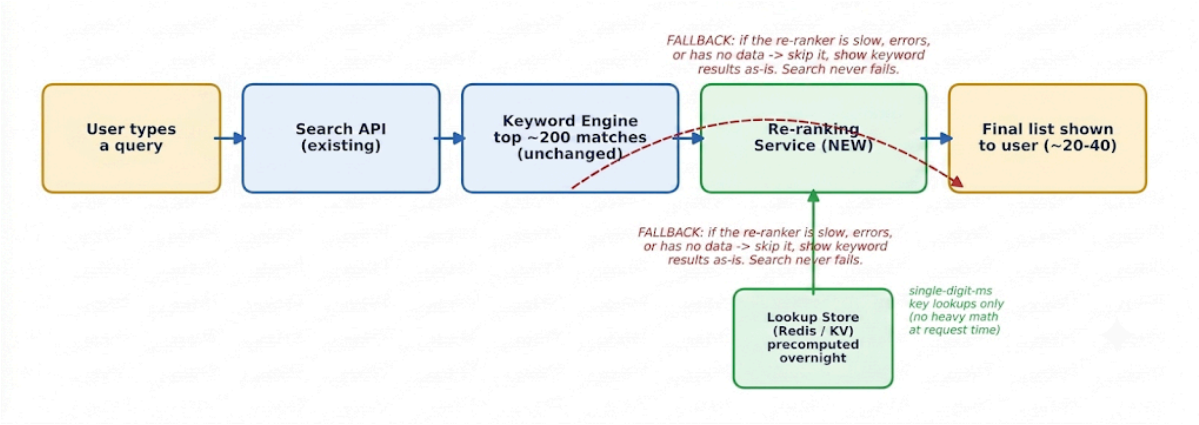
5.2 System Architecture

The proposed solution introduces three additional components while keeping the existing search infrastructure unchanged.

- **Nightly Batch Pipeline:** Processes historical search logs and order history to generate personalization features.
- **Redis Feature Store:** Stores the precomputed features for fast request-time retrieval.
- **Re-ranking Service:** Combines keyword relevance with personalization signals to compute the final ranking.

The existing Search API and Elasticsearch remain unchanged, reducing deployment risk and simplifying integration.

Diagram 3 : How One Search Request Flows Through the System



Source: *Diagram created by the author with assistance from Google Gemini*.

Design Note: The personalization layer operates independently of the retrieval system. If personalization is unavailable, the original keyword ranking is returned, ensuring high availability and minimal impact on search latency.

5.3 Request Flow

When a user submits a search query, the Search API forwards the request to Elasticsearch, which retrieves the top keyword-matched candidate products.

The Re-ranking Service then retrieves the required personalization features from Redis, computes the final ranking score for each candidate, and returns the re-ranked product list to the Search API.

If personalization data is unavailable or the re-ranking service cannot respond within the latency budget, the system simply returns the original Elasticsearch ranking.

5.4 Latency Considerations

The proposed architecture keeps the online request path lightweight.

The expensive operations, including feature generation and aggregation, are performed offline through nightly batch jobs. During request processing, only Redis lookups and simple score computation are required.

This ensures that personalization adds only a small amount of additional latency while maintaining the existing search performance.

5.5 Failure Handling

The personalization layer is designed to fail safely without affecting the core search functionality.

- Missing personalization features result in keyword-only ranking.
- Unknown or guest users use population-level signals and current-session interactions.
- If the re-ranking service is unavailable or exceeds the timeout, the Search API returns the original Elasticsearch results.
- If offline features become stale, they are ignored and the system falls back to keyword ranking.

This guarantees that search availability is never dependent on the personalization service.

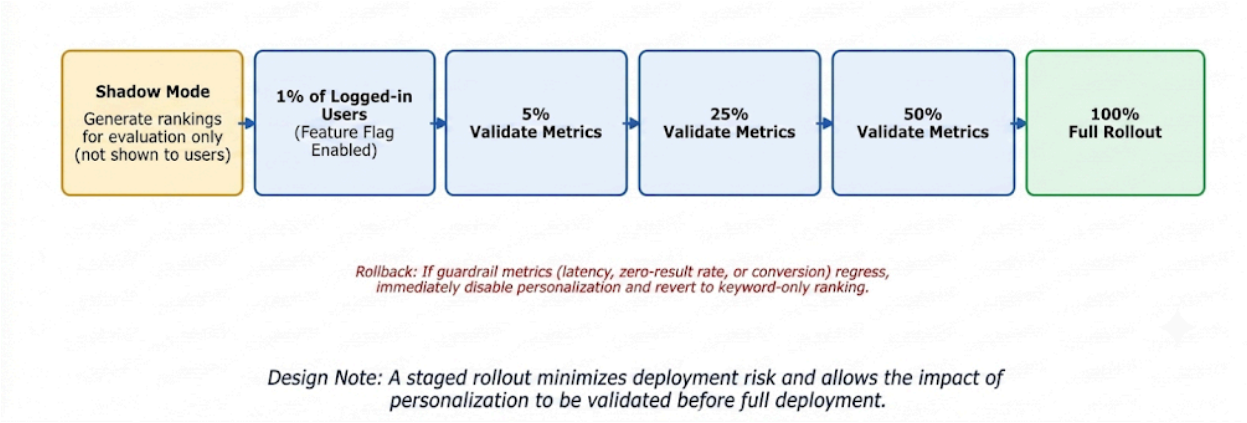
5.6 Rollout Strategy

The system should be deployed gradually using feature flags.

Deployment begins in **shadow mode**, where personalized rankings are generated but not shown to users. Once validated, the rollout can proceed through staged traffic percentages (1%, 5%, 25%, 50%, and finally 100%) while continuously monitoring business and system metrics.

If any guardrail metric degrades, the feature can be immediately disabled without affecting the existing search engine.

Diagram 3 : Staged Rollout Plan



Source: *Diagram created by the author with assistance from Google Gemini*.

5.7 Monitoring

The following metrics should be monitored after deployment.

Business Metrics

- Click-Through Rate (CTR)
- Conversion Rate
- Revenue per Search

System Metrics

- p95/p99 latency
- Redis cache hit rate
- Re-ranking service timeout rate
- Nightly batch job success rate

Quality Metrics

- Zero-result rate
- Result diversity
- Performance across new vs. returning users

Continuous monitoring ensures that personalization improves user experience without compromising system reliability.

6 Closing Thoughts

This design intentionally prioritizes simplicity over complexity. By building personalization on top of the existing keyword search engine, the solution delivers measurable improvements without introducing heavy infrastructure or complex machine learning pipelines.

The proposed architecture is explainable, easy to deploy, and resilient, with clear fallback mechanisms and minimal impact on request latency. It also creates a strong foundation for future enhancements, allowing more advanced ranking models to be introduced once sufficient production data and supporting infrastructure become available.

7 Appendix: AI Tools and References

1.How I Used AI Tools
As requested in the instructions, here is exactly how I used AI to help me with this assignment.

ChatGPT

- **What I used it for:** To help organize my ideas and write the text.
- **My prompt:** *"Help me write a simple system design document for personalized search using a Redis database."*
- **How I changed its answer:** ChatGPT suggested adding complex, real-time data streaming. I deleted those parts because the assignment asked for a simple solution.

Claude

- **What I used it for:** To help create the math example in the document.
- **My prompt:** *"Make a simple math table comparing a generic shoe and a Nike shoe using a search score formula."*\
  .

Google Gemini

- **What I used it for:** To plan what to draw in my system diagrams.
- **My prompt:** *"What boxes should I draw to show a simple search re-ranking system?"*

2.References and Sources

- **Elasticsearch Official Documentation:** Used to understand how basic keyword search and BM25 ranking works.
- **Redis Documentation:** Used to understand how to store pre-calculated data so it can be read very fast when a user searches.
- **"System Design Interview" by Alex Xu:** Used for general ideas on how to build simple and reliable web services without overcomplicating them.
- **Tech Blogs:** Read basic articles on how big e-commerce sites use simple counting and purchase history to suggest products.